Experiment No. 8

Student Name: Shubham Agarwal **UID: 25MCI10091**
Branch: MCA (AI & ML) **Section/Group: 25MAM-1/A**
Semester: 2nd **Date of Performance:12/03/2026**
Subject Name: Technical Training Lab **Subject Code: 25CAH-653**

Aim of the Session:

To apply the concept of Stored Procedures in database operations in order to perform tasks like insertion, updating, deletion, and retrieval of data efficiently, securely, and in a reusable manner within the database system.

Software Requirements:

- PostgreSQL Database Server
- pgAdmin 4
- Windows Operating System

Objective of the Session:

- To understand the concept and usage of stored procedures in database systems.
- To implement stored procedures for performing CRUD operations efficiently.
- To enhance data security by restricting direct access through stored procedures.
- To reduce code redundancy by using reusable database procedures.
- To improve performance of database operations using precompiled stored procedures

Practical Experiment Steps:

Table Creation:

```
CREATE TABLE employees (
   emp_id INT PRIMARY KEY,
   emp_name VARCHAR(50),
   department VARCHAR(50),
   designation VARCHAR(50),
   salary NUMERIC(20,3),
   joining_date DATE
);
```

Record Insertion:

```
INSERT INTO employees (emp_id, emp_name, department, designation, salary, joining_date) VALUES
(1, 'Rahul Sharma', 'IT', 'Software Engineer', 45000.000, '2022-03-15'),
(2, 'Priya Verma', 'HR', 'HR Manager', 55000.000, '2021-07-10'),
(3, 'Amit Gupta', 'Finance', 'Accountant', 40000.000, '2020-11-05'),
(4, 'Sneha Kapoor', 'IT', 'Frontend Developer', 42000.000, '2023-01-20'),
(5, 'Vikas Singh', 'Sales', 'Sales Executive', 38000.000, '2022-06-25'),
(6, 'Neha Joshi', 'Marketing', 'Marketing Manager', 60000.000, '2019-09-12'),
(7, 'Arjun Mehta', 'IT', 'Backend Developer', 47000.000, '2021-12-01');

SELECT * FROM employees;
```

	emp_id [PK] integer	emp_name character varying (50)	department character varying (50)	designation character varying (50)	salary numeric (20,3)	joining_date date
1	1	Rahul Sharma	IT	Software Engineer	45000.000	2022-03-15
2	2	Priya Verma	HR	HR Manager	55000.000	2021-07-10
3	3	Amit Gupta	Finance	Accountant	40000.000	2020-11-05
4	4	Sneha Kapoor	IT	Frontend Developer	42000.000	2023-01-20
5	5	Vikas Singh	Sales	Sales Executive	38000.000	2022-06-25
6	6	Neha Joshi	Marketing	Marketing Manager	60000.000	2019-09-12
7	7	Arjun Mehta	IT	Backend Developer	47000.000	2021-12-01

Step 1: Creating a Stored Procedure:

```
CREATE OR REPLACE procedure update_salary_proc(IN P_EMP_ID INT,INOUT P_SALARY
NUMERIC(20,3),OUT STATUS VARCHAR(20))
AS
$$
DECLARE
CURR_SAL NUMERIC(20,3);
BEGIN

SELECT SALARY+P_SALARY INTO CURR_SAL FROM employees WHERE EMP_ID=P_EMP_ID;
IF NOT FOUND THEN
RAISE EXCEPTION 'EMPLOYEE NOT FOUND';
END IF;

UPDATE employees
SET salary= curr_sal
where emp_id=p_emp_id;

P_salary:=curr_sal;
status:='SUCCESS';
```

```
EXCEPTION
WHEN OTHERS THEN
IF SQLERRM LIKE '%EMPLOYEE NOT FOUND%' THEN
STATUS:= 'EMPLOYEE NOT FOUND';
END IF;

END;
$$ LANGUAGE PLPGSQL;
```

```
CREATE PROCEDURE

Query returned successfully in 42 msec.
```

Step 2: Calling Stored Procedure

```
DO
        $$
        DECLARE
        EMP_ID INT:=1;
        STATUS VARCHAR(20);
        SALARY NUMERIC(20,3):=500;
        BEGIN
        CALL update_salary_proc(Emp_id,salary,status);
        RAISE NOTICE 'YOUR STATUS IS % AND YOUR UPDATED SALARY IS
%',STATUS,salary;
        END;
        $$
```

```
NOTICE:  YOUR STATUS IS SUCCESS AND YOUR UPDATED SALARY IS  45500.000
DO

Query returned successfully in 959 msec.
```

I/O Analysis

Input:

- Employee Data

Output:

- **Salary Update:** Adds increment to existing salary and updates database
- **Validation:** Checks if employee exists
- **Updated Salary:** Returned using `INOUT` parameter
- **Status Message:** Returns `SUCCESS` or `EMPLOYEE NOT FOUND`
- **Error Handling:** Handles exceptions properly

Learning Outcomes

- **Stored Procedure Implementation:**Students will be able to create and execute stored procedures using `IN`, `OUT`, and `INOUT` parameters for real-world database operations.
- **Error Handling & Validation:**Students will understand how to implement exception handling in PL/pgSQL to manage invalid inputs and unexpected conditions.
- **Logical Flow Understanding:**Students will gain knowledge of control structures like variable declaration, conditional statements, and query execution inside procedures.
- **Real-world Application:**Students will be able to design systems such as payroll management, employee management, or banking systems where automated updates and validations are required.